

Descripción de la Capa de Persistencia

Proyecto: Colegio O'Higgins Platform — Fullstack III (DSY1106)

1. Arquitectura de Datos

El sistema implementa una estrategia de **Database per Service**, garantizando el desacoplamiento y la independencia de escalabilidad entre microservicios. Cada servicio de dominio posee su propio esquema de base de datos, evitando acoplamiento en la capa de persistencia.

2. Componentes

Componente	Tecnología	Propósito
Motor BD	MySQL 8.0	Base de datos relacional
ORM	Spring Data JPA / Hibernate	Mapeo objeto-relacional
Migración	ddl-auto=update	Sincronización automática de entidades
Inicialización	Infra/mysql/init.sql	Creación de esquemas y usuarios con least privilege

3. Esquemas de Base de Datos

Base de Datos	Propósito	Acceso
colegio_auth_db	Gestión de usuarios, roles y autenticación JWT	API Gateway (Spring Security)
db_academic	Datos maestros de alumnos (RUT, nombre, curso, matrícula)	ms-students (CRUD estudiantes)
db_record	Registro histórico de asistencia (presente, atraso, inasistencia)	ms-attendance (gestión de asistencia)

4. Estrategia de Persistencia

- DDL automático:** Hibernate con `ddl-auto=update` crea y actualiza las tablas según las entidades JPA en cada microservicio.

- **Migraciones:** No se utiliza Flyway ni Liquibase; el esquema base se inicializa desde `init.sql` y Hibernate sincroniza el delta de entidades automáticamente.
- **Conexiones:** Cada microservicio configura su propio `DataSource` apuntando a su base de datos mediante variables de entorno (`DB_HOST` , `DB_PORT` , `DB_NAME`).
- **Transacciones:** `@Transactional` con aislamiento `READ_COMMITTED` . Se evitan transacciones distribuidas (2PC) para mantener la independencia de servicios y la escalabilidad horizontal.
- **Pool de conexiones:** HikariCP (incluido por defecto en Spring Boot) con timeouts configurables.

5. Seguridad

- Usuario de aplicación `app_colegio` con permisos mínimos (`SELECT`, `INSERT`, `UPDATE`, `DELETE`) en los 3 esquemas.
- Credenciales gestionadas mediante `Secrets` en Docker Compose y Kubernetes, no versionadas en el repositorio (`.env` enlistado en `.gitignore`).
- Cifrado: Conexiones JDBC con `useSSL=false` en desarrollo; se recomienda SSL/TLS en producción.

6. Script de Inicialización (init.sql)

El archivo `Infra/mysql/init.sql` ejecuta al arrancar el contenedor MySQL:

- Crea las 3 bases de datos con `CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci` .
- Concede permisos `SELECT, INSERT, UPDATE, DELETE, CREATE, ALTER, INDEX, DROP` al usuario `app_colegio` .
- Garantiza que el esquema está listo antes de que los microservicios intenten conectarse (via `depends_on` + `healthcheck` en Docker Compose).